

Web Programming Practice Set

20 Complete HTML + CSS + JS Mini-Projects, With Code and Explanations

Every project below is a full, working single-file program you can copy into a `.html` file and open directly in a browser -- no build tools, no installs. Each one combines structure (HTML), style (CSS) and behavior (JavaScript) together, the way real pages are built.

Read the code first, run it, then change something small and see what breaks. The "How it works" note under each project explains the key ideas so you're learning the concept, not just copying the code.

Projects are ordered Beginner → Intermediate → Advanced. Work through them in order for a natural progression, or jump to whichever topic you want to practice.

Contents

1. Personal ID Card	Beginner
2. Digital Business Card	Beginner
3. Recipe Card	Beginner
4. About Me / Profile Page	Beginner
5. Color Palette Generator	Beginner
6. Simple Calculator	Beginner
7. BMI Calculator	Beginner
8. Library Book Catalog	Intermediate
9. Student Grade Calculator	Intermediate
10. Quiz App	Intermediate
11. Expense Tracker	Intermediate
12. FAQ Accordion	Intermediate
13. Tabbed Content Viewer	Intermediate
14. Image Gallery with Lightbox	Intermediate
15. Countdown Timer	Advanced
16. Stopwatch	Advanced
17. Random Quote Generator	Advanced
18. Password Strength Checker	Advanced
19. Memory Matching Game	Advanced
20. Shopping Cart Mini App	Advanced

1. Personal ID Card

Beginner

Skills practiced: box model, flexbox, border-radius, box-shadow, image handling

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial,sans-serif;background:#e9edf2;display:flex;justify-content:center;padding:40px}
  .card{width:320px;background:#fff;border-radius:12px;box-shadow:0 8px 20px
rgba(0,0,0,.15);overflow:hidden;border:1px solid #ccc}
  .header{background:#2b6cb0;color:#fff;padding:14px;text-align:center;font-weight:bold}
  .photo{width:80px;height:80px;border-radius:50%;background:#2b6cb0;color:#fff;
    display:flex;align-items:center;justify-content:center;font-size:28px;margin:16px auto}
  .field{display:flex;justify-content:space-between;padding:4px 20px;font-size:14px}
  .field span:first-child{color:#666}
  .sign{margin:20px;border-top:1px solid #999;text-align:center;padding-top:4px;font-size:12px;color:#666}
</style>
</head>
<body>
<div class="card">
  <div class="header">GREENFIELD INSTITUTE</div>
  <div class="photo">AS</div>
  <div class="field"><span>Name</span><span>Aditi Sharma</span></div>
  <div class="field"><span>ID No.</span><span>GI-2026-0142</span></div>
  <div class="field"><span>DOB</span><span>01 Jan 2000</span></div>
  <div class="field"><span>Blood Group</span><span>O+</span></div>
  <div class="field"><span>Address</span><span>Kathmandu, NP</span></div>
  <div class="field"><span>Valid Till</span><span>Dec 2027</span></div>
  <div class="sign">Authorized Signature</div>
</div>
</body>
</html>
```

How it works: A pure structure-and-style exercise, no logic required. The outer `.card` is a flex-centered box with rounded corners and a soft box-shadow to fake real-world depth. The `.photo` circle shows how `border-radius:50%` turns a square div into a circle, and flexbox (`display:flex; align-items:center; justify-content:center`) centers the initials inside it perfectly. Each `.field` row uses `justify-content:space-between` so the label sits left and the value sits right without any manual spacing. This is the same layout pattern used in real ID cards, invoices, and settings pages -- label on one side, value on the other.

2. Digital Business Card

Beginner

Skills practiced: `classList.toggle()`, CSS transitions, flexbox, `target="_blank"` links

```
<!DOCTYPE html>
<html>
<head>
<style>
  :root{--bg:#fff;--text:#111}
  body.dark{--bg:#1a1a2e;--text:#f0f0f0}
  body{font-family:Arial,sans-serif;background:var(--bg);color:var(--text);
    display:flex;justify-content:center;align-items:center;height:100vh;margin:0;transition:.4s}
  .card{border:1px solid #888;border-radius:14px;padding:30px;width:260px;text-align:center;transition:.4s}
  .links{display:flex;justify-content:center;gap:14px;margin-top:14px}
  .links a{color:inherit;text-decoration:none;font-size:14px}
  button{margin-top:18px;padding:8px 16px;border:none;border-radius:
6px;background:#2b6cb0;color:#fff;cursor:pointer}
</style>
</head>
<body>
<div class="card">
  <h2>Aditi Sharma</h2>
  <p>Frontend Developer</p>
  <p style="font-size:13px">Builds clean, fast web experiences.</p>
  <div class="links">
    <a href="mailto:hi@example.com" target="_blank">Email</a>
    <a href="https://github.com" target="_blank">GitHub</a>
    <a href="https://linkedin.com" target="_blank">LinkedIn</a>
  </div>
  <button onclick="document.body.classList.toggle('dark')">Toggle Theme</button>
</div>
</body>
</html>
```

How it works: Introduces CSS custom properties (`--bg`, `--text`) as theme variables. The 'dark' class on simply overrides those two variables, and because every element references `var(--bg)` / `var(--text)`, the whole page repaints without touching any JS beyond one line: `classList.toggle('dark')`. The `transition:.4s` on both `body` and `.card` makes that swap animate smoothly instead of snapping instantly. `target="_blank"` on the contact links opens them in a new tab, standard practice for outbound links so users don't lose the current page.

3. Recipe Card

Beginner

Skills practiced: `<ol>/<ul>`, checkboxes, `window.print()`, `@media print`

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Georgia,serif;background:#faf6f0;display:flex;justify-content:center;padding:30px}
  .card{width:360px;background:#fff;border:1px solid #ddd;border-radius:8px;padding:20px}
  .meta{display:flex;justify-content:space-between;font-size:13px;color:#777;margin-bottom:10px}
  li{margin:4px 0}
  @media print{ button{display:none} }
</style>
</head>
<body>
<div class="card">
  <h2>Lemon Garlic Pasta</h2>
  <div class="meta"><span>Prep: 10m</span><span>Cook: 15m</span><span>Serves: 2</span></div>
  <h4>Ingredients</h4>
  <ul>
    <li><label><input type="checkbox"> 200g pasta</label></li>
    <li><label><input type="checkbox"> 2 cloves garlic</label></li>
    <li><label><input type="checkbox"> 1 lemon</label></li>
    <li><label><input type="checkbox"> Olive oil</label></li>
  </ul>
  <h4>Steps</h4>
  <ol>
    <li>Boil pasta until al dente.</li>
    <li>Sauté garlic in olive oil.</li>
    <li>Toss pasta with garlic, lemon zest and juice.</li>
  </ol>
  <button onclick="window.print()">Print Recipe</button>
</div>
</card>
</body>
</html>
```

How it works: Shows the difference between an unordered list (

, for the checklist, order doesn't matter) and an ordered list (

, for steps, order matters). Wrapping each checkbox in a means clicking the ingredient text also toggles the checkbox -- a small usability detail worth remembering. `window.print()` opens the browser's print dialog with zero JavaScript logic; the `@media print` rule then hides the button only when actually printing, which is how print-friendly pages are built without a second HTML file.

4. About Me / Profile Page

Beginner

Skills practiced: semantic tags (header, main, section, footer), flexbox wrapping, anchor targets

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial,sans-serif;margin:0;color:#222}
  header{background:#222;color:#fff;padding:40px 20px;text-align:center}
  main{max-width:600px;margin:30px auto;padding:0 20px}
  .tags{display:flex;flex-wrap:wrap;gap:8px}
  .tag{background:#e2e8f0;padding:6px 12px;border-radius:20px;font-size:13px}
  .links a{margin-right:12px}
  footer{text-align:center;padding:16px;color:#777;font-size:13px}
</style>
</head>
<body>
<header><h1>Aditi Sharma</h1><p>Web developer who loves clean UI</p></header>
<main>
  <section>
    <h3>Skills</h3>
    <div class="tags">
      <span class="tag">HTML</span><span class="tag">CSS</span>
      <span class="tag">JavaScript</span><span class="tag">React</span>
    </div>
  </section>
  <section class="links">
    <h3>Links</h3>
    <a href="#" target="_blank">GitHub</a>
    <a href="#" target="_blank">Portfolio</a>
  </section>
</main>
<footer>&copy; 2026 Aditi Sharma</footer>
</body>
</html>
```

How it works: Uses real semantic tags --

```
,
,
,
```

-- instead of generic

s, which is what screen readers and search engines expect from a real page. The .tags row is display:flex; flex-wrap:wrap, so pills reflow onto a new line automatically when the screen narrows, instead of overflowing or shrinking. No JavaScript at all here on purpose: the goal is to get comfortable structuring a page correctly before adding behavior.

5. Color Palette Generator

Beginner

Skills practiced: *Math.random()*, *string building*, *navigator.clipboard*, *inline style via JS*

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial,sans-serif;text-align:center;padding:40px}
  .row{display:flex;justify-content:center;gap:10px;margin:20px 0}
  .swatch{width:90px;height:120px;border-radius:8px;display:flex;align-items:flex-end;
    justify-content:center;color:#fff;font-size:13px;cursor:pointer;padding-bottom:8px}
  button{padding:10px 20px;border:none;border-radius:6px;background:#2b6cb0;color:#fff;cursor:pointer}
</style>
</head>
<body>
<h2>Color Palette Generator</h2>
<div class="row" id="row"></div>
<button onclick="generate()">Generate</button>
<script>
  const row = document.getElementById('row');
  function randColor(){
    return '#' + Math.floor(Math.random()*0xffffffff).toString(16).padStart(6,'0');
  }
  function generate(){
    row.innerHTML = '';
    for(let i=0;i<5;i++){
      const hex = randColor();
      const div = document.createElement('div');
      div.className = 'swatch';
      div.style.background = hex;
      div.textContent = hex;
      div.onclick = () => {
        navigator.clipboard.writeText(hex);
        div.textContent = 'Copied!';
        setTimeout(()=> div.textContent = hex, 800);
      };
      row.appendChild(div);
    }
  }
  generate();
</script>
</body>
</html>
```

How it works: The core trick is `randColor()`: `Math.random()*0xffffffff` picks a number between 0 and 16,777,215 (every possible RGB combination), `.toString(16)` converts it to hexadecimal, and `.padStart(6,'0')` guarantees six digits even when the number is small (e.g. 'a3' becomes '0000a3'). Each swatch's background is set directly via `div.style.background = hex`, showing how JS can drive CSS at runtime, not just at page load. `navigator.clipboard.writeText()` copies text without a hidden textarea trick, the modern way to implement "copy to clipboard."

6. Simple Calculator

Beginner

Skills practiced: event delegation on a button grid, state management in variables, expression evaluation

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{display:flex;justify-content:center;padding:30px;font-family:Arial}
  .calc{width:220px}
  #display{width:100%;height:50px;font-size:22px;text-align:right;margin-bottom:8px;
    box-sizing:border-box;padding:6px}
  .grid{display:grid;grid-template-columns:repeat(4,1fr);gap:6px}
  button{padding:14px;font-size:16px;border:none;border-radius:6px;background:#eee;cursor:pointer}
  button.op{background:#2b6cb0;color:#fff}
</style>
</head>
<body>
<div class="calc">
  <input id="display" readonly>
  <div class="grid" id="pad"></div>
</div>
<script>
  const display = document.getElementById('display');
  const pad = document.getElementById('pad');
  const keys = ['7','8','9','÷','4','5','6','×','1','2','3','-','C','0','.','+','='];
  let expr = '';
  keys.forEach(k => {
    const b = document.createElement('button');
    b.textContent = k;
    if('÷×-+'.includes(k)) b.classList.add('op');
    b.onclick = () => handle(k);
    pad.appendChild(b);
  });
  function handle(k){
    if(k === 'C'){ expr = ''; }
    else if(k === '='){
      try{
        const safe = expr.replace(/÷/g,'/').replace(/×/g,'*').replace(/-/g,'-');
        expr = String(eval(safe));
      }catch(e){ expr = 'Error'; }
    } else {
      expr += k;
    }
    display.value = expr;
  }
</script>
</body>
</html>
```

How it works: The button grid is built from an array of key labels in a loop rather than 17 hand-written

tags -- a common pattern once a UI has repeating elements. A single string variable, `expr`, holds the whole running expression as text (e.g. `"5+3*2"`); every keypress just appends to or evaluates that string. Because `+` and `*` aren't valid JS operators, they're swapped for `÷` and `×` right before `eval()` runs. `eval()` is used here purely for teaching simplicity since it directly demonstrates expression evaluation -- worth knowing that production calculators write a small parser instead of using `eval()`.

7. BMI Calculator

Beginner

Skills practiced: `parseFloat`, conditional logic, `.toFixed()`, form validation

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;display:flex;justify-content:center;padding:40px}
  .box{width:280px}
  input{width:100%;padding:8px;margin:6px 0;box-sizing:border-box}
  button{width:100%;padding:10px;background:#2b6cb0;color:#fff;border:none;border-radius:6px;cursor:pointer}
  #result{margin-top:14px;font-weight:bold}
  #error{color:#c53030;font-size:13px}
</style>
</head>
<body>
<div class="box">
  <h2>BMI Calculator</h2>
  <input id="height" type="number" placeholder="Height (cm)">
  <input id="weight" type="number" placeholder="Weight (kg)">
  <button onclick="calc()">Calculate</button>
  <div id="error"></div>
  <div id="result"></div>
</div>
<script>
  function calc(){
    const h = parseFloat(document.getElementById('height').value) / 100;
    const w = parseFloat(document.getElementById('weight').value);
    const err = document.getElementById('error');
    const res = document.getElementById('result');
    if(!h || !w || h <= 0 || w <= 0){
      err.textContent = 'Enter valid positive numbers.';
      res.textContent = '';
      return;
    }
    err.textContent = '';
    const bmi = (w / (h*h)).toFixed(1);
    let cat = 'Normal';
    if(bmi < 18.5) cat = 'Underweight';
    else if(bmi >= 25 && bmi < 30) cat = 'Overweight';
    else if(bmi >= 30) cat = 'Obese';
    res.textContent = `BMI: ${bmi} (${cat})`;
  }
</script>
</body>
</html>
```

How it works: `parseFloat()` turns the text from an

into a real number (inputs are always strings until you convert them). The guard clause -- `if(!h || !w || h<=0 || w<=0)` return -- is the standard shape for form validation: check for bad input first and bail out early, so the rest of the function can assume the data is clean. `.toFixed(1)` rounds the result to one decimal for display while keeping the underlying number precise. The if/else-if chain that follows is a direct translation of the BMI category boundaries into code, a pattern that repeats constantly in real apps (grading scales, pricing tiers, shipping zones).

8. Library Book Catalog

Intermediate

Skills practiced: `array.filter()`, `array.map()`, dynamic DOM rendering, combining two filters, input event

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;padding:30px;max-width:600px;margin:auto}
  .controls{display:flex;gap:10px;margin-bottom:16px}
  input,select{padding:8px;flex:1}
  .book{border:1px solid #ddd;border-radius:6px;padding:10px;margin-bottom:8px;
    display:flex;justify-content:space-between}
  .badge{padding:2px 8px;border-radius:12px;font-size:12px;color:#fff}
  .yes{background:#2f855a}.no{background:#c53030}
</style>
</head>
<body>
<h2>Library Catalog</h2>
<div class="controls">
  <input id="search" placeholder="Search title or author...">
  <select id="category"><option value="">All Categories</option></select>
</div>
<div id="list"></div>
<script>
  const books = [
    {title:'Clean Code', author:'Robert Martin', category:'Tech', available:true},
    {title:'Sapiens', author:'Yuval Harari', category:'Non-fiction', available:false},
    {title:'Dune', author:'Frank Herbert', category:'Fiction', available:true},
    {title:'Cosmos', author:'Carl Sagan', category:'Science', available:true},
    {title:'1984', author:'George Orwell', category:'Fiction', available:false},
    {title:'The Selfish Gene', author:'Richard Dawkins', category:'Science', available:true},
    {title:'Educated', author:'Tara Westover', category:'Non-fiction', available:true},
    {title:'Atomic Habits', author:'James Clear', category:'Non-fiction', available:false},
    {title:'The Hobbit', author:'J.R.R. Tolkien', category:'Fiction', available:true},
    {title:'A Brief History of Time', author:'Stephen Hawking', category:'Science', available:true}
  ];
  const list = document.getElementById('list');
  const search = document.getElementById('search');
  const category = document.getElementById('category');
  [...new Set(books.map(b=>b.category))].forEach(c=>{
    const o = document.createElement('option'); o.value=c; o.textContent=c;
    category.appendChild(o);
  });
  function render(){
    const q = search.value.toLowerCase();
    const c = category.value;
    const filtered = books.filter(b =>
      (b.title.toLowerCase().includes(q) || b.author.toLowerCase().includes(q)) &&
      (c === '' || b.category === c)
    );
    list.innerHTML = filtered.map(b => `
      <div class="book">
        <span><b>${b.title}</b> by ${b.author} <small>(${b.category})</small></span>
        <span class="badge ${b.available ? 'yes' : 'no'}">${b.available ? 'Available' : 'Checked Out'}</span>
      </div>`).join('') || '<p>No books found.</p>';
  }
  search.addEventListener('input', render);
  category.addEventListener('change', render);
  render();
</script>
```

```
</body>
</html>
```

How it works: This is the anchor project for "treat data as an array of objects" thinking. The books array is the single source of truth; nothing is hard-coded into the HTML. The category dropdown is built dynamically too: `books.map(b=>b.category)` pulls out every category, and wrapping it in `new Set(...)` removes duplicates, so adding an

`.render()` is the heart of the project -- it re- v

runs the filter every time the search box fires an 'input' event or the dropdown fires 'change', combining two conditions (text match AND category match) in one `.filter()` call, then rebuilds the list with `.map()` and a template literal. This filter-then-render pattern is exactly how real search-and-filter UIs

11th book never requires touching the work.

9. Student Grade Calculator

Intermediate

Skills practiced: array of objects as state, building a <table> dynamically, array.reduce(), grade conditionals

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;padding:30px;max-width:500px;margin:auto}
  .form{display:flex;gap:8px;margin-bottom:16px}
  input{padding:8px;flex:1}
  table{width:100%;border-collapse:collapse}
  td,th{border:1px solid #ddd;padding:6px;text-align:left}
  #summary{margin-top:12px;font-weight:bold}
</style>
</head>
<body>
<h2>Grade Calculator</h2>
<div class="form">
  <input id="subject" placeholder="Subject">
  <input id="marks" type="number" placeholder="Marks /100">
  <button onclick="addRow()">Add</button>
</div>
<table>
  <thead><tr><th>Subject</th><th>Marks</th><th></th></tr></thead>
  <tbody id="rows"></tbody>
</table>
<div id="summary"></div>
<script>
  let subjects = [];
  function addRow(){
    const name = document.getElementById('subject').value.trim();
    const marks = parseFloat(document.getElementById('marks').value);
    if(!name || isNaN(marks)) return;
    subjects.push({name, marks});
    document.getElementById('subject').value = '';
    document.getElementById('marks').value = '';
    render();
  }
  function remove(i){ subjects.splice(i,1); render(); }
  function render(){
    document.getElementById('rows').innerHTML = subjects.map((s,i) => `
      <tr><td>${s.name}</td><td>${s.marks}</td>
      <td><button onclick="remove(${i})">Remove</button></td></tr>` ).join('');
    const total = subjects.reduce((sum,s) => sum + s.marks, 0);
    const avg = subjects.length ? (total / subjects.length) : 0;
    let grade = 'F';
    if(avg >= 90) grade = 'A';
    else if(avg >= 75) grade = 'B';
    else if(avg >= 60) grade = 'C';
    else if(avg >= 40) grade = 'D';
    document.getElementById('summary').textContent =
      subjects.length ? `Total: ${total} | Average: ${avg.toFixed(1)} | Grade: ${grade}` : '';
  }
</script>
</body>
</html>
```

How it works: subjects is the app's "state" -- one JS array that everything else is derived from. Every change (add or remove) mutates that array and then calls render() once, which rebuilds the table rows AND recalculates the totals in the same pass. array.reduce() collapses the whole array into one number (the total) by carrying a running sum

through each item -- a cleaner alternative to a manual for-loop with a counter variable. `splice(i,1)` removes exactly one item at index `i` from the array, which is how the Remove button deletes a row without disturbing the others.

10. Quiz App

Intermediate

Skills practiced: current-question index, array indexing, conditional styling, resetting state

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;display:flex;justify-content:center;padding:40px}
  .box{width:340px}
  .opt{display:block;width:100%;padding:10px;margin:6px 0;border:1px solid #ccc;
    border-radius:6px;background:#fff;cursor:pointer;text-align:left}
  .correct{background:#c6f6d5;border-color:#2f855a}
  .wrong{background:#fed7d7;border-color:#c53030}
</style>
</head>
<body>
<div class="box" id="app"></div>
<script>
  const questions = [
    {q:'What does HTML stand for?',
      options:['Hyper Trainer Marking Language','Hyper Text Markup Language',
        'Hyper Text Marketing Language','Hyperlinks Text Markup Language'], correct:1},
    {q:'Which tag makes text bold?', options:['<bold>','<b>','<strong-text>','<bd>'], correct:1},
    {q:'CSS stands for?',
      options:['Cascading Style Sheets','Creative Style System',
        'Computer Style Sheets','Colorful Style Sheets'], correct:0},
    {q:'Which keyword declares a constant in JS?', options:['var','let','const','static'], correct:2},
    {q:'Which array method adds an item to the end?', options:['push()','add()','append()','insert()'],
correct:0}
  ];
  let i = 0, score = 0;
  const app = document.getElementById('app');
  function render(){
    if(i >= questions.length){
      app.innerHTML = `<h2>You scored ${score} out of ${questions.length}</h2><button
onclick="restart()">Retry</button>`;
      return;
    }
    const q = questions[i];
    app.innerHTML = `<h3>${q.q}</h3>` + q.options.map((o,idx) =>
      `<button class="opt" onclick="answer(${idx})">${o}</button>`).join('');
  }
  function answer(idx){
    const q = questions[i];
    const btns = document.querySelectorAll('.opt');
    btns[q.correct].classList.add('correct');
    if(idx !== q.correct) btns[idx].classList.add('wrong');
    else score++;
    btns.forEach(b => b.onclick = null);
    setTimeout(() => { i++; render(); }, 800);
  }
  function restart(){ i = 0; score = 0; render(); }
  render();
</script>
</body>
</html>
```

How it works: Two plain numbers -- `i` (current question index) and `score` -- are the entire state of this app. `render()` always draws whatever `questions[i]` currently is, so advancing the quiz is just "`i++` then `render()` again", never a full page reload. `answer()` compares the clicked index to `q.correct`, colors the right button green and, if wrong, the clicked

one red, disables every button so a second click can't change the answer, then waits 800ms with `setTimeout` before moving on so the feedback is actually visible. `restart()` resets both state variables to zero, which is the whole "reset a mini-app" pattern used again in the Memory Game.

11. Expense Tracker

Intermediate

Skills practiced: array push/filter, live recalculation, currency formatting

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;padding:30px;max-width:480px;margin:auto}
  .form{display:flex;gap:6px;margin-bottom:16px}
  input,select{padding:8px}
  .item{display:flex;justify-content:space-between;border:1px solid #ddd;border-radius:6px;
    padding:8px;margin-bottom:6px}
  .tag{background:#edf2f7;padding:2px 8px;border-radius:10px;font-size:12px}
  #total{font-weight:bold;font-size:18px;margin-top:10px}
</style>
</head>
<body>
<h2>Expense Tracker</h2>
<div class="form">
  <input id="name" placeholder="Item">
  <input id="amount" type="number" placeholder="Amount">
  <select id="category"><option>Food</option><option>Travel</option><option>Bills</option><option>Other</
option></select>
  <button onclick="add()">Add</button>
</div>
<div id="list"></div>
<div id="total"></div>
<script>
  let expenses = [];
  function add(){
    const name = document.getElementById('name').value.trim();
    const amount = parseFloat(document.getElementById('amount').value);
    const category = document.getElementById('category').value;
    if(!name || isNaN(amount)) return;
    expenses.push({name, amount, category});
    document.getElementById('name').value = '';
    document.getElementById('amount').value = '';
    render();
  }
  function remove(i){ expenses.splice(i,1); render(); }
  function render(){
    document.getElementById('list').innerHTML = expenses.map((e,i) => `
      <div class="item">
        <span>${e.name}</span> <span class="tag">${e.category}</span></div>
        <span>${e.amount.toFixed(2)} <button onclick="remove(${i})">x</button></span>
      </div>`).join('');
    const total = expenses.reduce((s,e) => s + e.amount, 0);
    document.getElementById('total').textContent = `Total: ${total.toFixed(2)}`;
  }
</script>
</body>
</html>
```

How it works: Structurally almost identical to the Grade Calculator -- an array holding objects, one render() function that redraws the list and recomputes a total on every change. What's new here is treating it as a running total that must never go stale: because render() is called after every add() and every remove(), the total line is always in sync with the array, instead of being calculated once and forgotten. .toFixed(2) formats every amount as currency with exactly two decimals, which is worth doing consistently anywhere money is shown.

12. FAQ Accordion

Intermediate

Skills practiced: `classList.toggle()`, CSS transitions on `max-height`, looping through siblings

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;max-width:500px;margin:40px auto}
  .item{border-bottom:1px solid #ddd}
  .q{display:flex;justify-content:space-between;padding:14px 4px;cursor:pointer;font-weight:bold}
  .a{max-height:0;overflow:hidden;transition:max-height .3s ease;font-size:14px;color:#444}
  .a p{margin:0 0 12px}
  .plus{transition:.3s}
  .open .plus{transform:rotate(45deg)}
</style>
</head>
<body>
<h2>FAQs</h2>
<div id="faq"></div>
<script>
  const data = [
    {q:'What is this course about?', a:'A hands-on introduction to HTML, CSS and JavaScript.'},
    {q:'Do I need prior experience?', a:'No, it starts from the basics.'},
    {q:'How long does it take?', a:'About 2-3 weeks at a relaxed pace.'},
    {q:'Is there a certificate?', a:'Yes, after completing all practice sets.'},
    {q:'Can I ask questions?', a:'Yes, always feel free to ask.'}
  ];
  const faq = document.getElementById('faq');
  data.forEach(item => {
    const div = document.createElement('div');
    div.className = 'item';
    div.innerHTML = `<div class="q">${item.q}<span class="plus">+</span></div><div class="a"><p>${item.a}</p></div>`;
    const q = div.querySelector('.q');
    const a = div.querySelector('.a');
    q.onclick = () => {
      const isOpen = div.classList.contains('open');
      document.querySelectorAll('.item').forEach(i => {
        i.classList.remove('open'); i.querySelector('.a').style.maxHeight = null;
      });
      if(!isOpen){ div.classList.add('open'); a.style.maxHeight = a.scrollHeight + 'px'; }
    };
    faq.appendChild(div);
  });
</script>
</body>
</html>
```

How it works: The expand/collapse animation is a CSS trick, not a JS one: `.a` starts at `max-height:0` with `overflow:hidden`, and JS only ever sets `a.style.maxHeight = a.scrollHeight + 'px'` (its real content height). Because `max-height` has a transition, the browser animates between those two values automatically. The "only one open at a time" behavior comes from looping through every `.item` with `forEach` and closing all of them before opening the clicked one -- a very common pattern any time a UI needs mutually-exclusive open/closed sections (dropdown menus, sidebar filters, etc).

13. Tabbed Content Viewer

Intermediate

Skills practiced: `querySelectorAll` + `forEach`, `data-*` attributes, toggling an active class

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;max-width:500px;margin:40px auto}
  .tabs{display:flex;border-bottom:2px solid #ddd}
  .tab{padding:10px 16px;cursor:pointer;color:#666}
  .tab.active{color:#2b6cb0;border-bottom:2px solid #2b6cb0;margin-bottom:-2px}
  .panel{display:none;padding:16px 4px}
  .panel.active{display:block}
</style>
</head>
<body>
<div class="tabs">
  <div class="tab active" data-tab="about">About</div>
  <div class="tab" data-tab="skills">Skills</div>
  <div class="tab" data-tab="projects">Projects</div>
  <div class="tab" data-tab="contact">Contact</div>
</div>
<div class="panel active" id="about">I'm a web developer learning by building small projects.</div>
<div class="panel" id="skills">HTML, CSS, JavaScript, and growing.</div>
<div class="panel" id="projects">20 mini-projects from this practice set.</div>
<div class="panel" id="contact">Reach me at hi@example.com</div>
<script>
  document.querySelectorAll('.tab').forEach(tab => {
    tab.addEventListener('click', () => {
      document.querySelectorAll('.tab').forEach(t => t.classList.remove('active'));
      document.querySelectorAll('.panel').forEach(p => p.classList.remove('active'));
      tab.classList.add('active');
      document.getElementById(tab.dataset.tab).classList.add('active');
    });
  });
</script>
</body>
</html>
```

How it works: `data-tab="about"` is a custom `data-*` attribute -- HTML's built-in way to attach a small piece of information to an element without inventing a new class or ID scheme. In JS, `tab.dataset.tab` reads that value back and matches it directly to a panel's id, so clicking a tab needs no if/else chain of "if this tab, show that panel." `querySelectorAll(...).forEach` is the standard way to grab every matching element and loop over it; here it's used twice -- once to clear `.active` from every tab and panel, then again to add `.active` back to only the one that was clicked and its matching panel.

14. Image Gallery with Lightbox

Intermediate

Skills practiced: CSS Grid, position: fixed overlays, z-index stacking, dynamic event listeners

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;padding:30px;max-width:600px;margin:auto}
  .grid{display:grid;grid-template-columns:repeat(4,1fr);gap:8px}
  .grid img{width:100%;height:80px;object-fit:cover;border-radius:6px;cursor:pointer}
  .overlay{display:none;position:fixed;inset:0;background:rgba(0,0,0,.85);
    align-items:center;justify-content:center;flex-direction:column}
  .overlay.show{display:flex}
  .overlay img{max-width:80%;max-height:70vh;border-radius:6px}
  .overlay button{margin-top:16px;padding:8px 16px;border:none;border-radius:6px;cursor:pointer}
  .close{position:absolute;top:20px;right:30px;color:#fff;font-size:24px;cursor:pointer}
</style>
</head>
<body>
<h2>Gallery</h2>
<div class="grid" id="grid"></div>
<div class="overlay" id="overlay">
  <span class="close" onclick="close_()">&times;</span>
  <img id="big">
  <div>
    <button onclick="nav(-1)">Prev</button>
    <button onclick="nav(1)">Next</button>
  </div>
</div>
<script>
  // Replace these with your own image paths/URLs.
  const urls = Array.from({length:8}, (_,i) => `img${i+1}.jpg`);
  let current = 0;
  const grid = document.getElementById('grid');
  urls.forEach((u,i) => {
    const img = document.createElement('img');
    img.src = u;
    img.onclick = () => open_(i);
    grid.appendChild(img);
  });
  function open_(i){
    current = i;
    document.getElementById('big').src = urls[i];
    document.getElementById('overlay').classList.add('show');
  }
  function close_(){ document.getElementById('overlay').classList.remove('show'); }
  function nav(dir){
    current = (current + dir + urls.length) % urls.length;
    document.getElementById('big').src = urls[current];
  }
  document.getElementById('overlay').addEventListener('click', e => {
    if(e.target.id === 'overlay') close_();
  });
</script>
</body>
</html>
```

How it works: The thumbnail grid is plain CSS Grid: `grid-template-columns:repeat(4,1fr)` creates four equal columns and the browser wraps rows automatically as images are added. The lightbox itself is `position:fixed; inset:0`, which pins it to the full viewport regardless of scroll position -- the standard way to build any full-screen overlay (modals,

cookie banners, image viewers). `nav(dir)` demonstrates wraparound indexing: `(current + dir + urls.length) % urls.length` keeps the index inside the array bounds in both directions, so "Prev" on the first image safely jumps to the last one. Clicking the dark backdrop closes the overlay by checking `e.target.id === 'overlay'` -- i.e. "was the click on the background itself, not the image inside it." Swap the placeholder `img1.jpg...img8.jpg` paths for real image files or URLs.

15. Countdown Timer

Advanced

Skills practiced: the Date object, setInterval, converting milliseconds into days/hours/minutes/seconds

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;text-align:center;padding:60px}
  .box{display:flex;gap:20px;justify-content:center;font-size:32px}
  .unit{display:flex;flex-direction:column}
  .unit span:last-child{font-size:12px;color:#666}
</style>
</head>
<body>
<h2>Countdown to New Year</h2>
<div class="box" id="box"></div>
<script>
  const target = new Date('2027-01-01T00:00:00');
  const box = document.getElementById('box');
  function update(){
    const diff = target - new Date();
    if(diff <= 0){ box.innerHTML = "<h3>Time's up!</h3>"; clearInterval(timer); return; }
    const d = Math.floor(diff / 86400000);
    const h = Math.floor(diff / 3600000) % 24;
    const m = Math.floor(diff / 60000) % 60;
    const s = Math.floor(diff / 1000) % 60;
    box.innerHTML = [[d,'Days'],[h,'Hrs'],[m,'Min'],[s,'Sec']]
      .map(([v,l]) => `<div class="unit"><span>${v}</span><span>${l}</span></div>`).join('');
  }
  update();
  const timer = setInterval(update, 1000);
</script>
</body>
</html>
```

How it works: `target - new Date()` subtracts two `Date` objects and JavaScript quietly converts both to milliseconds since 1970, giving `diff` as a plain number of milliseconds remaining. Everything below that is unit conversion: dividing by 86,400,000 (ms in a day) gives days; dividing by 3,600,000 gives total hours, and `% 24` trims that down to "hours within the current day," the same `%` pattern repeats for minutes and seconds. `setInterval(update, 1000)` re-runs `update()` every second, which is how any "live" clock, timer, or auto-refreshing widget works. Once `diff` drops to zero or below, `clearInterval(timer)` stops the loop so it doesn't keep running forever in the background.

16. Stopwatch

Advanced

Skills practiced: `setInterval/clearInterval`, timestamp math with `Date.now()`, dynamically building a list

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;text-align:center;padding:60px}
  #time{font-size:40px;margin-bottom:16px}
  button{padding:10px 16px;margin:4px;border:none;border-radius:
6px;background:#2b6cb0;color:#fff;cursor:pointer}
  ul{list-style:none;padding:0;max-width:200px;margin:16px auto;text-align:left}
</style>
</head>
<body>
<div id="time">00:00.00</div>
<button onclick="start()">Start</button>
<button onclick="pause()">Pause</button>
<button onclick="reset()">Reset</button>
<button onclick="lap()">Lap</button>
<ul id="laps"></ul>
<script>
  let startTime = 0, elapsed = 0, timer = null;
  function format(ms){
    const m = String(Math.floor(ms/60000)).padStart(2,'0');
    const s = String(Math.floor(ms/1000)%60).padStart(2,'0');
    const cs = String(Math.floor(ms/10)%100).padStart(2,'0');
    return `${m}:${s}.${cs}`;
  }
  function tick(){
    document.getElementById('time').textContent = format(Date.now() - startTime + elapsed);
  }
  function start(){
    if(timer) return;
    startTime = Date.now();
    timer = setInterval(tick, 30);
  }
  function pause(){
    if(!timer) return;
    elapsed += Date.now() - startTime;
    clearInterval(timer); timer = null;
  }
  function reset(){
    clearInterval(timer); timer = null; elapsed = 0;
    document.getElementById('time').textContent = '00:00.00';
    document.getElementById('laps').innerHTML = '';
  }
  function lap(){
    const li = document.createElement('li');
    li.textContent = document.getElementById('time').textContent;
    document.getElementById('laps').appendChild(li);
  }
</script>
</body>
</html>
```

How it works: The one subtlety worth studying here: time is tracked with `Date.now()` timestamps, not by counting interval ticks. `start()` records the exact moment it began; every tick computes `Date.now() - startTime + elapsed`, i.e. "how long since we last pressed start, plus whatever was already banked from before." `pause()` folds the current

running duration into elapsed and stops the interval, so resuming later continues from the right total instead of restarting at zero -- this timestamp-based approach stays accurate even if the tab is throttled and setInterval ticks aren't perfectly on time, unlike a naive counter that just adds 1 on every tick. lap() simply reads whatever the display currently shows and appends it as a new

- .

17. Random Quote Generator

Advanced

Skills practiced: `Math.random()` with array indexing, avoiding repeats, the Clipboard API

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Georgia,serif;display:flex;justify-content:center;padding:60px}
  .box{max-width:400px;text-align:center}
  blockquote{font-size:20px;margin:0 0 8px}
  cite{color:#666}
  button{margin:6px;padding:8px 16px;border:none;border-radius:
6px;background:#2b6cb0;color:#fff;cursor:pointer}
</style>
</head>
<body>
<div class="box">
  <blockquote id="text"></blockquote>
  <cite id="author"></cite><br>
  <button onclick="newQuote()">New Quote</button>
  <button onclick="copy()">Copy</button>
</div>
<script>
  const quotes = [
    {text:'Simplicity is the soul of efficiency.', author:'Austin Freeman'},
    {text:'Code is like humor. When you have to explain it, it is bad.', author:'Cory House'},
    {text:'First, solve the problem. Then, write the code.', author:'John Johnson'},
    {text:'Programs must be written for people to read.', author:'Harold Abelson'},
    {text:'Make it work, make it right, make it fast.', author:'Kent Beck'},
    {text:'The best error message is the one that never shows up.', author:'Thomas Fuchs'},
    {text:'Talk is cheap. Show me the code.', author:'Linus Torvalds'},
    {text:'Experience is the name everyone gives to their mistakes.', author:'Oscar Wilde'},
    {text:'Fix the cause, not the symptom.', author:'Steve Maguire'},
    {text:'Good code is its own best documentation.', author:'Steve McConnell'}
  ];
  let last = -1;
  function newQuote(){
    let i;
    do { i = Math.floor(Math.random() * quotes.length); } while(i === last);
    last = i;
    document.getElementById('text').textContent = quotes[i].text;
    document.getElementById('author').textContent = '– ' + quotes[i].author;
  }
  function copy(){
    navigator.clipboard.writeText(document.getElementById('text').textContent);
  }
  newQuote();
</script>
</body>
</html>
```

How it works: `Math.floor(Math.random() * quotes.length)` is the standard formula for "a random valid index into this array" -- worth memorizing since it comes up constantly (random image, random enemy spawn, random background). The `do...while` loop is what prevents the exact same quote from showing twice in a row: it keeps rerolling `i` until it differs from `last`, then remembers the new choice for next time. `copy()` reuses the same `navigator.clipboard.writeText()` call from the Color Palette Generator, showing how the same small API shows up across very different projects once you've learned it once.

18. Password Strength Checker

Advanced

Skills practiced: regular expressions, the input event, dynamically updating classes/styles per keystroke

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;display:flex;justify-content:center;padding:60px}
  .box{width:300px}
  input{width:100%;padding:8px;box-sizing:border-box}
  ul{list-style:none;padding:0;font-size:13px}
  li{margin:4px 0;color:#c53030}
  li.ok{color:#2f855a}
  .bar{height:8px;border-radius:4px;background:#eee;margin-top:8px}
  .fill{height:100%;border-radius:4px;width:0;transition:.3s;background:#c53030}
  #label{font-size:13px;margin-top:4px}
</style>
</head>
<body>
<div class="box">
  <h3>Password Strength</h3>
  <input id="pw" type="password" placeholder="Enter password">
  <button onclick="toggle()">Show/Hide</button>
  <ul id="checks">
    <li id="len">At least 8 characters</li>
    <li id="up">One uppercase letter</li>
    <li id="low">One lowercase letter</li>
    <li id="num">One number</li>
    <li id="sp">One special character</li>
  </ul>
  <div class="bar"><div class="fill" id="fill"></div></div>
  <div id="label"></div>
</div>
<script>
const pw = document.getElementById('pw');
function toggle(){ pw.type = pw.type === 'password' ? 'text' : 'password'; }
pw.addEventListener('input', () => {
  const v = pw.value;
  const rules = {
    len: v.length >= 8,
    up: /[A-Z]/.test(v),
    low: /[a-z]/.test(v),
    num: /[0-9]/.test(v),
    sp: /^[^A-Za-z0-9]/.test(v)
  };
  let score = 0;
  for(const key in rules){
    document.getElementById(key).classList.toggle('ok', rules[key]);
    if(rules[key]) score++;
  }
  const fill = document.getElementById('fill');
  const label = document.getElementById('label');
  const pct = (score / 5) * 100;
  fill.style.width = pct + '%';
  if(score <= 2){ fill.style.background = '#c53030'; label.textContent = 'Weak'; }
  else if(score <= 4){ fill.style.background = '#dd6b20'; label.textContent = 'Medium'; }
  else { fill.style.background = '#2f855a'; label.textContent = 'Strong'; }
});
</script>
```

```
</body>
</html>
```

How it works: Each rule is a small regular expression tested against the current input: `/[A-Z]/.test(v)` asks "does v contain at least one uppercase letter anywhere," and `/[^A-Za-z0-9]/` (a negated character class) asks "does v contain any character that is NOT a letter or digit," which is a compact way to detect punctuation/symbols. The listener is attached to the 'input' event rather than 'change', so it fires on every keystroke instead of only when the field loses focus -- essential for real-time feedback. `classList.toggle('ok', rules[key])` is the two-argument form of toggle: it explicitly adds 'ok' when the condition is true and removes it when false, rather than flipping it blindly.

19. Memory Matching Game

Advanced

Skills practiced: shuffling an array, setTimeout, tracking game state, CSS transforms

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;text-align:center;padding:30px}
  .grid{display:grid;grid-template-columns:repeat(4,70px);gap:8px;justify-content:center;margin:20px auto}
  .card{width:70px;height:70px;background:#2b6cb0;border-radius:8px;display:flex;
    align-items:center;justify-content:center;font-size:26px;color:transparent;
    cursor:pointer;transition:.2s}
  .card.flip{color:#fff;background:#4a5568}
  .card.matched{background:#2f855a;color:#fff}
  #status{margin-top:10px}
</style>
</head>
<body>
<h2>Memory Match</h2>
<div id="status">Moves: 0</div>
<div class="grid" id="grid"></div>
<script>
  const emojis = [' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' '];
  let cards = [...emojis, ...emojis].sort(() => Math.random() - 0.5);
  let flipped = [], matched = [], moves = 0;
  const grid = document.getElementById('grid');
  cards.forEach((val, i) => {
    const div = document.createElement('div');
    div.className = 'card';
    div.textContent = val;
    div.onclick = () => flip(i, div);
    grid.appendChild(div);
  });
  function flip(i, div){
    if(flipped.length === 2 || div.classList.contains('flip') || matched.includes(i)) return;
    div.classList.add('flip');
    flipped.push({i, div});
    if(flipped.length === 2){
      moves++;
      document.getElementById('status').textContent = `Moves: ${moves}`;
      const [a, b] = flipped;
      if(cards[a.i] === cards[b.i]){
        a.div.classList.add('matched'); b.div.classList.add('matched');
        matched.push(a.i, b.i);
        flipped = [];
        if(matched.length === cards.length){
          document.getElementById('status').textContent = `You Won! Moves: ${moves}`;
        }
      } else {
        setTimeout(() => {
          a.div.classList.remove('flip'); b.div.classList.remove('flip');
          flipped = [];
        }, 700);
      }
    }
  }
</script>
</body>
</html>
```

How it works: [...emojis, ...emojis] duplicates the 8-emoji array into 16 items (8 matching pairs), and .sort() => Math.random() - 0.5) is a quick-and-common (if imperfect) way to shuffle an array by giving the sort comparator a random result each time. Three plain variables carry the whole game's state: flipped (the 0, 1 or 2 cards currently face-up in this turn), matched (every index that's been permanently solved), and moves (a simple counter). The guts of flip() are its guard clause -- refusing to flip a third card, a card that's already face-up, or one that's already matched -- and the setTimeout(...,700) that gives the player 700ms to actually see a wrong pair before it flips back, which is what makes it feel like a real memory game instead of an instant flash.

20. Shopping Cart Mini App

Advanced

Skills practiced: array of objects as cart state, quantity math, conditional rendering, currency formatting

```
<!DOCTYPE html>
<html>
<head>
<style>
  body{font-family:Arial;padding:30px;max-width:600px;margin:auto}
  .products{display:grid;grid-template-columns:repeat(3,1fr);gap:10px}
  .p{border:1px solid #ddd;border-radius:6px;padding:10px;text-align:center}
  .p button{margin-top:6px;padding:6px 10px;border:none;border-radius:
4px;background:#2b6cb0;color:#fff;cursor:pointer}
  .cart{margin-top:20px;border-top:2px solid #ddd;padding-top:10px}
  .row{display:flex;justify-content:space-between;align-items:center;margin:6px 0}
  .row button{margin:0 2px}
  #total{font-weight:bold;font-size:18px}
</style>
</head>
<body>
<h2>Shop</h2>
<div class="products" id="products"></div>
<div class="cart">
  <h3>Cart</h3>
  <div id="cartList"></div>
  <div id="total"></div>
  <button onclick="clearCart()">Clear Cart</button>
</div>
<script>
  const products = [
    {id:1, name:'T-Shirt', price:15},
    {id:2, name:'Mug', price:8},
    {id:3, name:'Notebook', price:5},
    {id:4, name:'Sticker Pack', price:3},
    {id:5, name:'Cap', price:12},
    {id:6, name:'Bag', price:20}
  ];
  let cart = [];
  const pDiv = document.getElementById('products');
  products.forEach(p => {
    const div = document.createElement('div');
    div.className = 'p';
    div.innerHTML = `<b>${p.name}</b><br>${p.price}<br><button onclick="addToCart(${p.id})">Add to Cart</
button>`;
    pDiv.appendChild(div);
  });
  function addToCart(id){
    const item = cart.find(c => c.id === id);
    if(item) item.qty++;
    else cart.push({...products.find(p => p.id === id), qty: 1});
    render();
  }
  function changeQty(id, delta){
    const item = cart.find(c => c.id === id);
    item.qty += delta;
    if(item.qty <= 0) cart = cart.filter(c => c.id !== id);
    render();
  }
  function clearCart(){ cart = []; render(); }
  function render(){
    const list = document.getElementById('cartList');
    list.innerHTML = cart.length ? cart.map(c => `
```

```

<div class="row">
  <span>${c.name} x${c.qty}</span>
  <span>
    <button onclick="changeQty(${c.id}, -1)">-</button>
    <button onclick="changeQty(${c.id}, 1)">+</button>
    ${ (c.price * c.qty).toFixed(2) }
  </span>
</div>`).join('') : '<p>Cart is empty.</p>';
const total = cart.reduce((s,c) => s + c.price * c.qty, 0);
document.getElementById('total').textContent = `Total: ${total.toFixed(2)}`;
}
render();
</script>
</body>
</html>

```

How it works: The most "app-like" project in the set, and a good capstone because it reuses almost every earlier idea at once: an array of objects (cart), .find() to locate one item by id, .filter() to remove an item, .reduce() to total the price times quantity, and render() as the single function that keeps the DOM in sync with the array. addToCart() shows a useful spread-operator trick -- {...products.find(p => p.id === id), qty: 1} copies all of a product's fields into a new object and adds a qty field on top, without mutating the original products array. changeQty() shows how deleting on a threshold works: if a decremented item's qty drops to zero or below, it's filtered out of the cart entirely rather than shown as "0 items."